ERIC's Practice Problem Generator - Backpropagation & Computation Graphs

EECS 182 Special Participation E

Part 1: The System Prompt

Copy this entire prompt into ChatGPT, Claude, or Gemini to generate practice problems:

You are an expert deep learning instructor creating practice problems for EECS 182 (graduate-level deep learning course).
Generate 6 practice problems on Backpropagation and Computation Graphs with the following structure:

REQUIREMENTS:
1. Problems should range from basic (warm-up) to advanced (exam-level)
2. Each problem must include:
   - Clear problem statement
   - Step-by-step solution showing ALL intermediate steps
   - Final numerical answer (when applicable)
   - Key insight or common mistake to avoid
3. Cover different aspects:
   - Basic chain rule application
   - Computation graphs with multiple paths
   - Vector/matrix derivatives
   - Tricky cases (shared parameters, multiple uses of same variable)
   - Numerical gradient checking
   - Architectural implications

PROBLEM DIFFICULTY PROGRESSION:
- Problem 1: Basic chain rule warm-up
- Problem 2: Simple computation graph
- Problem 3: Multiple paths in computation graph
- Problem 4: Vector/matrix derivatives
- Problem 5: Shared parameters / tricky case
- Problem 6: Architectural understanding or numerical checking

FORMAT FOR EACH PROBLEM:

Problem [N]: [Title]
Difficulty: [Basic/Intermediate/Advanced]

[Problem statement with clear notation]

Solution:
[Step-by-step solution showing work]

Key Insight:
[Important takeaway or common mistake]

Numerical Answer: [if applicable]

IMPORTANT:

- Use clear mathematical notation
- Show every step in the chain rule
- Explain why each step is taken
- For computation graphs, draw them in ASCII or describe clearly
- Include dimensions for vectors/matrices
- Point out common errors students make

Generate 6 problems now on Backpropagation and Computation Graphs.

Part 2: Example Usage - Problems Generated Using This Prompt

I used the above prompt with Claude to generate the following practice problems for studying backpropagation:

~~Problem 1: Chain Rule Warm-up~~ ~~Difficulty: Basic~~

Given the function $f(x) = (3x^2 + 2)^3$, compute df/dx at $x = 1$.

Solution: Using chain rule: $df/dx = df/du \cdot du/dx$

Let $u = 3x^2 + 2$
Then $f = u^3$

Step 1: Compute du/dx
$du/dx = 6x$

Step 2: Compute df/du
$df/du = 3u^2$

Step 3: Apply chain rule
$df/dx = 3u^2 \cdot 6x = 18xu^2$

Step 4: Substitute $u = 3x^2 + 2$
$df/dx = 18x(3x^2 + 2)^2$

Step 5: Evaluate at $x = 1$
$u(1) = 3(1)^2 + 2 = 5$
$df/dx|_{x=1} = 18(1)(5)^2 = 18 \cdot 25 = 450$

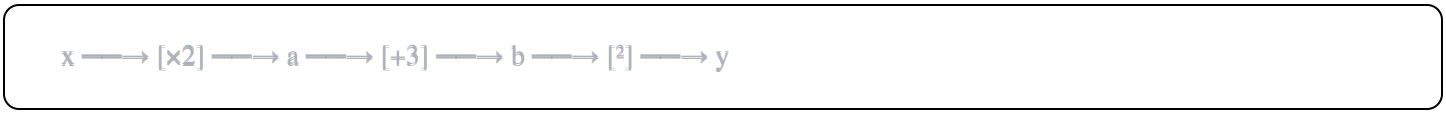
Key Insight: Always identify intermediate variables and work systematically. Chain rule flows from output to input.

Numerical Answer: 450

[ANNOTATION]: Good warm-up problem. Clear step-by-step breakdown. However, it might be more pedagogically valuable to also show the computation graph explicitly, even for this simple case, to build the visual intuition. Also, could mention that this is exactly what happens in a single neuron: linear transformation followed by activation.

Problem 2: Simple Computation Graph Difficulty: Basic

Consider the computation graph:



Given x = 2, compute dy/dx using backpropagation.

Solution: Forward pass:

- $a = 2x = 2(2) = 4$
- $b = a + 3 = 4 + 3 = 7$
- $y = b^2 = 7^2 = 49$

Backward pass (compute gradients):

- $dy/dy = 1$ (starting point)
- $dy/db = dy/dy \cdot \partial y/\partial b = 1 \cdot 2b = 2(7) = 14$
- $dy/da = dy/db \cdot \partial b/\partial a = 14 \cdot 1 = 14$
- $dy/dx = dy/da \cdot \partial a/\partial x = 14 \cdot 2 = 28$

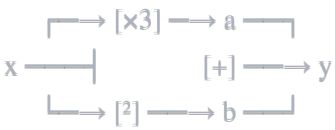
Key Insight: Backpropagation flows right-to-left through the graph, multiplying local gradients. Each node passes gradients to its inputs by multiplying by the local derivative.

Numerical Answer: dy/dx = 28

[ANNOTATION]: Good introduction to the computational graph concept and the backward pass. The step-by-step forward then backward is clear. However, the problem could be improved by explicitly showing that dy/da is computed as the sum of gradients from all paths (even though there's only one path here). This would set up Problem 3 better. Also, could mention that this is O(1) memory per operation vs O(n) for naive symbolic differentiation.

Problem 3: Multiple Paths in Computation Graph Difficulty: Intermediate

Consider this computation graph where x is used twice:



Given x = 2, compute dy/dx.

Solution: Forward pass:

- $a = 3x = 3(2) = 6$
- $b = x^2 = 2^2 = 4$
- $y = a + b = 6 + 4 = 10$

Backward pass:

- $dy/dy = 1$
- $dy/da = dy/dy \cdot \partial y/\partial a = 1 \cdot 1 = 1$
- $dy/db = dy/dy \cdot \partial y/\partial b = 1 \cdot 1 = 1$

Now x contributes to y through TWO paths:

- Path 1 (through a): dy/dx via $a = dy/da \cdot \partial a/\partial x = 1 \cdot 3 = 3$
- Path 2 (through b): dy/dx via $b = dy/db \cdot \partial b/\partial x = 1 \cdot 2x = 1 \cdot 2(2) = 4$

Total gradient: $dy/dx = 3 + 4 = 7$

Key Insight: When a variable appears multiple times in the computation graph, its gradient is the SUM of gradients from all paths. This is the multivariate chain rule in action.

Numerical Answer: $dy/dx = 7$

[ANNOTATION]: Excellent problem! This is where students often make mistakes - forgetting to sum gradients from multiple paths. The visualization of two paths is clear. One improvement: could explicitly verify this using the closed form $y = 3x + x^2 \rightarrow dy/dx = 3 + 2x = 3 + 4 = 7$. This would reinforce that backprop gives the correct mathematical derivative. Could also mention this is why we use "+=" in code implementations, not "=".

Problem 4: Vector/Matrix Derivatives Difficulty: Intermediate

Given:

- $x \in \mathbb{R}^3$ (input vector)
- $W \in \mathbb{R}^{2x3}$ (weight matrix)
- $b \in \mathbb{R}^2$ (bias vector)
- $y = Wx + b$ (linear layer output)
- $L = \|y\|^2 = y^T y$ (squared L2 norm loss)

Compute $\partial L/\partial W$ and $\partial L/\partial x$.

Solution: Forward pass dimensions:

- $y = Wx + b \rightarrow \mathbb{R}^2$ (2D output)
- $L = y^T y \rightarrow$ scalar

Backward pass:
Step 1: Compute $\partial L/\partial y$
$L = y^T y = \Sigma_i y_i^2$
$\partial L/\partial y = 2y$ (vector of dimension $\mathbb{R}^2$)

Step 2: Compute $\partial L/\partial W$
$y = Wx + b$, so $\partial y/\partial W$ involves the input x
For the i-th row of W (w_i): $\partial y_i/\partial w_i = x^T$

Using chain rule:
$\partial L/\partial W = \partial L/\partial y \cdot \partial y/\partial W = (2y)\ x^T$

This is an outer product: $\mathbb{R}^2 \otimes \mathbb{R}^3 \rightarrow \mathbb{R}^{2x3}$ (same shape as W)

Step 3: Compute $\partial L/\partial x$
$\partial y/\partial x = W$ (the weight matrix)
$\partial L/\partial x = \partial L/\partial y \cdot \partial y/\partial x = W^T\ (2y)$

This gives a $\mathbb{R}^3$ vector (same shape as x)

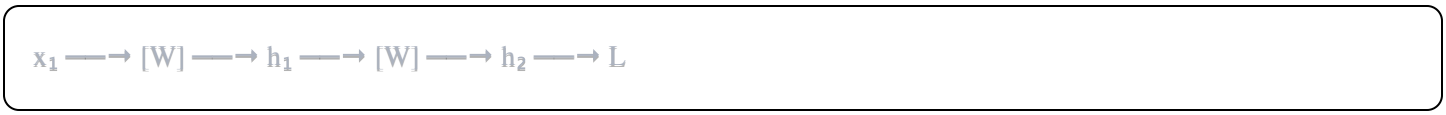
Key Insight: For linear layers:

- Gradient w.r.t. weights: outer product of upstream gradient and input
- Gradient w.r.t. input: matrix-vector product with transposed weights Always check dimensions match!

[ANNOTATION]: This is where the problem gets real - going from scalar calculus to vector/matrix calculus. The dimension checking is excellent and should always be emphasized. However, there's a missed opportunity here: could show a concrete numerical example with specific values for x, W, b to make this less abstract. Students often struggle with the abstraction. Also, could mention that this is exactly what torch.autograd computes under the hood, and why transposes appear everywhere in backprop code.

Problem 5: Shared Parameters (Tricky Case) Difficulty: Advanced

Consider a weight-sharing scenario common in RNNs:



Where:

- $h_1 = W x_1$ (same W used twice)
- $h_2 = W h_1 = W(W x_1) = W^2 x_1$
- $L = \|h_2\|^2$

The same weight matrix W is applied twice. Compute ∂L/∂W.

Solution: This is tricky because W appears in TWO places in the computational path.

Forward pass:

- $h_1 = W x_1$
- $h_2 = W h_1 = W^2 x_1$
- $L = h_2$^T h_2

Backward pass:
Step 1: $\partial L/\partial h_2 = 2h_2$

Step 2: ∂L/∂W from the second application of W
$h_2 = W h_1$, so $\partial h_2/\partial W = h_1$^T (treating h_1 as constant)
Contribution 1: $\partial L/\partial W|_2 = (2h_2)\ h_1$^T

Step 3: First we need $\partial L/\partial h_1$
$\partial L/\partial h_1 = \partial L/\partial h_2 \cdot \partial h_2/\partial h_1 = (2h_2)$^T $W = 2W$^T h_2

Step 4: $\partial L/\partial W$ from the first application of W
$h_1 = W x_1$, so $\partial h_1/\partial W = x_1^T$ (treating x_1 as constant)
Contribution 2: $\partial L/\partial W|_1 = (2W^T h_2) x_1^T$

Step 5: Total gradient (sum both contributions)
$\partial L/\partial W = \partial L/\partial W|_2 + \partial L/\partial W|_1 = (2h_2)h_1^T + (2W^T h_2)x_1^T$

Key Insight: When the same parameter appears multiple times in the computational graph, you must sum the gradients from ALL occurrences. This is crucial in RNNs where weights are shared across time steps. Missing one contribution leads to incorrect gradients.

Numerical Answer: $\partial L/\partial W = 2h_2h_1^T + 2(W^T h_2)x_1^T$

[ANNOTATION]: Excellent problem that captures a real source of confusion! This is exactly what happens in RNNs during backpropagation through time (BPTT). The solution correctly identifies both contributions. However, the problem could be strengthened by: (1) providing a small numerical example (say W is 2×2, x_1 is 2D) to show concretely that summing matters, and (2) explicitly mentioning that this is why gradients "accumulate" in RNN training and why we see "+=" operations in framework code. Also worth noting: this is a simple 2-step case, but in BPTT with T steps, you'd have T contributions to sum.

Problem 6: Numerical Gradient Checking Difficulty: Advanced

You've implemented backpropagation for a neural network layer. To verify correctness, you need to numerically check gradients.

Given:

- Function: $f(x) = (x_1^2 + x_2)^3$ where $x = [x_1, x_2]^T$
- Point: $x = [2, 3]^T$
- Analytical gradient: You computed $\nabla f = [\partial f/\partial x_1, \partial f/\partial x_2]^T$

a) Derive the analytical gradient ∇f
b) Compute the numerical gradient using finite differences with $\varepsilon = 0.0001$
c) Compute the relative error between analytical and numerical gradients

Solution:

Part (a): Analytical gradient
Let $u = x_1^2 + x_2$, then $f = u^3$

$\partial f/\partial x_1 = \partial f/\partial u \cdot \partial u/\partial x_1 = 3u^2 \cdot 2x_1 = 6x_1u^2$
$\partial f/\partial x_2 = \partial f/\partial u \cdot \partial u/\partial x_2 = 3u^2 \cdot 1 = 3u^2$

Analytical gradient: $\nabla f = [588, 147]^T$

Part (b): Numerical gradient
Using centered finite difference: $(f(x + \varepsilon) - f(x - \varepsilon)) / (2\varepsilon)$

For $\partial f/\partial x_1$ (perturb x_1):
$x^+ = [2.0001, 3]$, $x^- = [1.9999, 3]$
$f(x^+) = ((2.0001)^2 + 3)^3 = (7.0004)^3 \approx 343.4116$
$f(x^-) = ((1.9999)^2 + 3)^3 = (6.9996)^3 \approx 342.8820$
$\partial f/\partial x_1 \approx (343.4116 - 342.8820)/(2 \cdot 0.0001) = 0.5296/0.0002 \approx 587.98$

For $\partial f/\partial x_2$ (perturb x_2):
$x^+ = [2, 3.0001]$, $x^- = [2, 2.9999]$
$f(x^+) = (4 + 3.0001)^3 = (7.0001)^3 \approx 343.0294$
$f(x^-) = (4 + 2.9999)^3 = (6.9999)^3 \approx 342.9706$
$\partial f/\partial x_2 \approx (343.0294 - 342.9706)/(2 \cdot 0.0001) = 0.0588/0.0002 \approx 147.00$

Numerical gradient: $\nabla f_{num} \approx [587.98, 147.00]^T$

Part (c): Relative error
For each component:
Relative error = $|analytical - numerical| / \max(|analytical|, |numerical|)$

Component 1: $|588 - 587.98| / 588 = 0.02/588 \approx 0.000034 = 0.0034\%$
Component 2: $|147 - 147.00| / 147 \approx 0.000000 \approx 0\%$

Overall relative error (using L2 norm):
$\|\nabla f - \nabla f_{num}\|_2 / \|\nabla f\|_2 = \|(0.02, 0.00)\|_2 / \|(588, 147)\|_2 \approx 0.02/605.3 \approx 0.000033$

Key Insight: Gradient checking should give relative error $< 10^{-5}$ (excellent) or $< 10^{-3}$ (acceptable). Errors $> 10^{-2}$ indicate bugs in backprop implementation. Always use centered finite differences (not one-sided) for better accuracy. This is essential during debugging but too expensive for training.

Numerical Answer: Analytical: [588, 147], Numerical: [587.98, 147.00], Relative error: ~0.003%

[ANNOTATION]: This is the most practical problem in the set - gradient checking is something every DL practitioner needs to know. The solution correctly shows centered differences and relative error computation. Excellent inclusion of the rule of thumb ($< 10^{-5}$ excellent, $< 10^{-3}$ acceptable, $> 10^{-2}$ indicates bugs). However, some improvements: (1) Could mention why we use relative error instead of absolute error (scale invariance), (2) Should warn that gradient checking doesn't work well near zero or with non-smooth functions (ReLU, max

pooling), (3) Could mention that modern frameworks like PyTorch have torch.autograd.gradcheck() built-in. Overall this is an essential skill that many students skip.

Part 3: Overall Critical Assessment

What Works Well:

1. **Progressive difficulty**: Problems 1-2 are accessible warm-ups, 3-6 are exam-level
2. **Key insights**: Each problem highlights a common mistake or important concept
3. **Computational graphs**: Visual/ASCII representations help understanding
4. **Dimension checking**: Problem 4 emphasizes this crucial debugging skill
5. **Real scenarios**: Problems 3 and 5 reflect actual neural network situations

What Could Be Improved:

1. **Lack of numerical examples**: Several problems stay abstract when concrete numbers would help
2. **Missing connections to code**: Could mention how these concepts map to PyTorch/TensorFlow
3. **No ReLU/activation functions**: All problems use smooth functions; real networks have non-smooth activations
4. **Could add computational complexity**: Mention why backprop is O(n) not $O(n^2)$
5. **Problem 5 notation**: Could be clearer about W appearing in different positions vs. same W being reused

Suggested Additional Problems:

- Backprop through ReLU (handling the kink at 0)
- Backprop through max pooling (discrete selection)
- Batch normalization gradients (more complex multivariate chain rule)
- Gradient flow in ResNets (why skip connections help)

Overall Assessment: This is a solid problem set that builds from fundamentals to advanced cases. The progression is logical and covers the key conceptual hurdles in backpropagation. With some numerical examples and code connections, this would be an excellent study resource. The gradient checking problem (#6) is particularly valuable as it's often overlooked in courses.

Difficulty Rating:

- Problems 1-2: Good for HW warm-ups
- Problems 3-4: Typical exam questions
- Problems 5-6: Challenging exam questions or deeper understanding

---

Part 4: How to Use This System Prompt

For Your Classmates:

1. **Copy the System Prompt** (from Part 1) into your favorite AI assistant:
    - Claude (claude.ai) - tends to be more detailed
    - ChatGPT (chat.openai.com) - good for quick generation
    - Gemini (gemini.google.com) - alternative perspective
2. **Customize for Your Needs:**
    - Change the topic: "Generate problems on Attention Mechanisms"
    - Adjust difficulty: "Make all problems exam-level difficulty"
    - Focus area: "Focus on vector/matrix calculus in backprop"
    - Add constraints: "Include at least 2 problems with ReLU"
3. **Generate and Verify:**
    - Always check solutions yourself
    - Use gradient checking (Problem 6 method) for numerical problems
    - Compare with solutions manual or homework problems
    - If something seems wrong, ask the AI to explain its reasoning
4. **Iterate:**
    - "Problem 3 was too easy, make it harder"
    - "Show me a variant of Problem 5 with 3 time steps"
    - "Give me a problem combining Problems 3 and 4"

Example Customizations:

```
// For midterm prep
"Generate 6 problems on Backpropagation suitable for a 90-minute midterm exam.
Include 2 basic, 3 intermediate, 1 advanced. Focus on computation graphs and
the multivariate chain rule."

// For specific weak areas
"Generate 4 problems specifically on the case where a variable appears multiple
times in a computation graph (like Problem 3). Make them progressively harder."

// For homework help
"Generate 3 problems similar to EECS 182 HW4 Problem 2, but with different
functions and dimensions. Show full solutions."
```

Tips for Effective Use:

- ✅ Generate problems AFTER attempting homework to test understanding
- ✅ Do problems under timed conditions to simulate exams
- ✅ Work with study group and compare solutions
- ✅ Create your own variants of problems you got wrong
- ❌ Don't just read solutions - work through them yourself first
- ❌ Don't use for homework (that's academic dishonesty)

Pro Tip: After generating problems, ask the AI: "What are 3 common mistakes students make on these types of problems?" This helps you avoid pitfalls on exams.

Conclusion

This practice problem generator is a tool for active learning. The system prompt can be adapted to any topic in EECS 182 (Transformers, CNNs, Optimization, etc.) and serves as a foundation for generating unlimited practice material. The key is to always engage critically with generated content - verify solutions, identify gaps, and iterate.

Remember: The goal isn't to collect problems, it's to BUILD UNDERSTANDING. Use this system prompt to generate problems, struggle with them, check your work, and learn from mistakes. That's how you prepare for the final exam.

Good luck with your studying!

--- ERIC's submission for Special Participation E (Option E, submission #2)